

# Big Data Project

**Stream Clustering using Spark and MOA**

Authored by:

Hesam Kouchehi | 403723987

Hanieh Noorizade | 403724032

Supervisor:

Dr. Naderi

Fall 2024 | Tehran, Iran

Iran University of Science and Technology

---

# *Contents*

|                                         |    |
|-----------------------------------------|----|
| Abstract.....                           | 2  |
| Introduction .....                      | 3  |
| What is the problem.....                | 4  |
| Workflow .....                          | 5  |
| Dataset .....                           | 7  |
| Splitting.....                          | 9  |
| Train test .....                        | 9  |
| Stream.....                             | 9  |
| Visualizing data (RapidMiner) .....     | 10 |
| Training the model (Offline Phase)..... | 13 |
| Feature Selection.....                  | 13 |
| Standardize Features .....              | 13 |
| Training .....                          | 14 |
| Evaluation.....                         | 15 |
| Saving the Weights .....                | 16 |
| Stream Clusterer (Online Phase).....    | 17 |
| Loading the weights .....               | 17 |
| Defining CSV schema .....               | 17 |
| Making connection for Streamer.....     | 17 |
| Streamer program .....                  | 17 |
| Cluster the streams.....                | 18 |
| Evaluation.....                         | 19 |
| Tune the model.....                     | 20 |
| MOA.....                                | 21 |
| Conclusion .....                        | 22 |

## *Abstract*

In this project, we address the challenge of clustering streaming data using k-means clustering, focusing on real-time updates and dynamic performance evaluation. Unlike traditional batch processing methods, we develop a system that can handle continuously incoming data while maintaining accurate clustering performance. The project is built using an extended version of the Iris dataset, which is split into training and streaming components to simulate real-world data flow. An offline model is first trained using selected features and standardized data to initialize cluster centers, and a streaming model is subsequently used to dynamically assign clusters and update them as new data arrives. We evaluate the model's performance using metrics such as accuracy, recall, F1-score, and silhouette score. Our results demonstrate robust clustering performance with an average silhouette score of 0.61 and accuracy exceeding 95%. The system is designed to be scalable and adaptable, making it suitable for real-time applications like anomaly detection and pattern discovery in dynamic environments.

# *Introduction*

In the real world, data generation is a continuous process, driven by sources such as IoT sensors, website interactions, financial transactions, and online services. This type of continuously generated data, known as streaming data, poses a challenge for traditional data analysis methods, which rely on static, fixed datasets. As data keeps arriving, waiting for the entire dataset to be collected before performing analysis is not feasible in many real-time applications. There is a growing need for systems that can process and analyze data as it streams in, identifying patterns, tracking changes, and detecting anomalies in real time.

In this project, we tackle this problem by implementing a real-time clustering system using k-means clustering with the PySpark framework. The goal is to develop a system that can dynamically group incoming data into clusters and update these clusters as new data arrives. By maintaining up-to-date cluster assignments, the system will always reflect the current state of the data, making it useful for applications such as anomaly detection, dynamic pattern recognition, and real-time monitoring. The extended Iris dataset is used as a benchmark, providing a richer and larger dataset for effective training and testing of the system in both offline and online phases. Through proper feature selection, standardization, and evaluation, we demonstrate how k-means clustering can be adapted for real-time data processing while maintaining high accuracy and reliability.

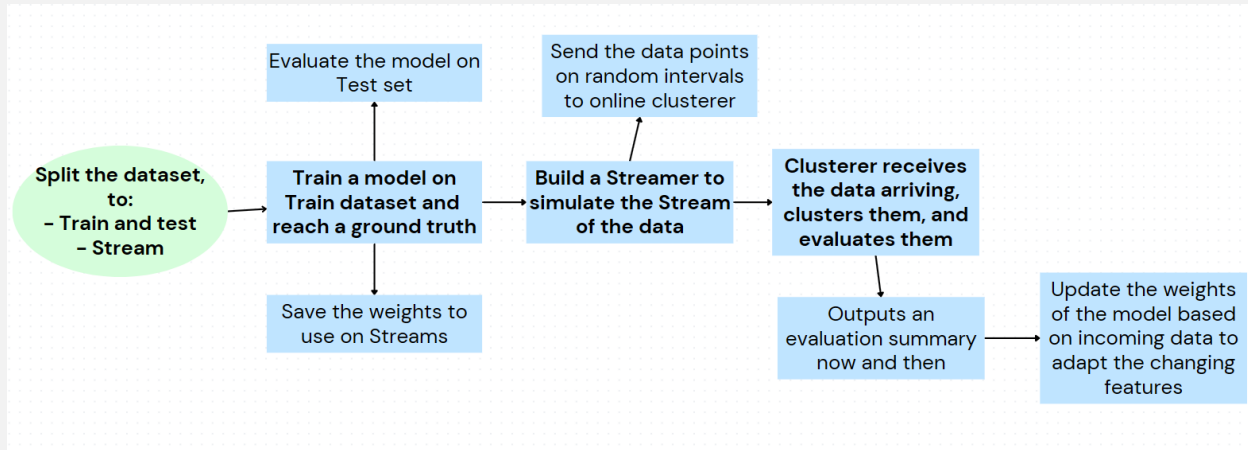
## *What is the problem*

In the real world, data is always changing. Data is generated constantly, like sensor readings, website clicks, or online purchases. This type of data is called streaming data because it keeps coming in over time. The problem is that traditional methods of analyzing data work with fixed datasets that don't change. But when data is streaming, it keeps updating, and we need a way to analyze it quickly without waiting for everything to be collected. We need a system that can group this data while it's still streaming and find useful patterns as the data changes.

In this project, we will use a method called k-means clustering. It groups similar data into clusters. Our system will take streaming data, group it into clusters, and update those clusters as new data arrives. This way, the system will always reflect the current state of the data. We will focus on making sure the clustering works in real time, so the system can be used for things like detecting patterns, tracking changes, or spotting unusual events right when they happen.

# Workflow

Here is a chart of what we have done:



The workflow of this project is divided into two main phases: the offline training phase and the online streaming phase. In the offline phase, the extended Iris dataset is split into a train-test set (80%) and a streaming set (20%) using the `splitter.py` file. The offline phase begins with selecting meaningful features to reduce dimensionality and improve computational efficiency. The selected features are then standardized using the `StandardScaler` to ensure that all features contribute equally to the clustering process. A `k-means` clustering model is trained on the scaled features, with the number of clusters set to 3 based on prior analysis using the silhouette score. The model is evaluated offline, achieving a silhouette score of 0.6174, and the resulting trained model, along with the feature assembler and scaler, is saved for the online phase.

In the online streaming phase, the saved offline model is loaded, and the system is initialized to handle continuously incoming data. A stream simulator, implemented in `streamer.py`, sends data points at random intervals to simulate real-time data streams. The online model processes these incoming batches, assigns clusters to the data points, and dynamically updates cluster centers to adapt to changing patterns. Evaluation metrics such as accuracy, recall, F1-score, and silhouette score are calculated in real time to monitor performance. To prevent sudden shifts in cluster assignments due to poor-quality batches, the model applies adaptive updating strategies like learning rate adjustments. This ensures that the clustering process remains stable and robust, even under

continuously changing conditions, making the system suitable for real-world applications involving dynamic data streams.

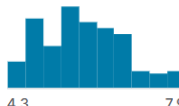
We will discuss each section in detail, further in this report.

## Dataset

In this project, we use the **Iris Extended**<sup>1</sup> dataset from Kaggle platform, which is an extended version of the classic Iris dataset. The original Iris dataset contains 150 samples of iris flowers, with measurements for sepal length, sepal width, petal length, and petal width, along with their corresponding species (Setosa, Versicolor, and Virginica). A brief summary of iris classic is shown at this image:

| summary | sepal_length       | sepal_width         | petal_length       | petal_width        | species   |
|---------|--------------------|---------------------|--------------------|--------------------|-----------|
| count   | 150                | 150                 | 150                | 150                | 150       |
| mean    | 5.843333333333335  | 3.0540000000000007  | 3.7586666666666693 | 1.1986666666666672 | null      |
| stddev  | 0.8280661279778637 | 0.43359431136217375 | 1.764420419952262  | 0.7631607417008414 | null      |
| min     | 4.3                | 2.0                 | 1.0                | 0.1                | setosa    |
| max     | 7.9                | 4.4                 | 6.9                | 2.5                | virginica |

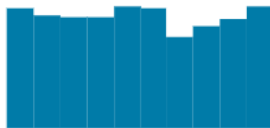
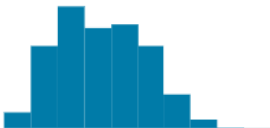
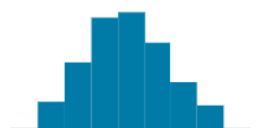
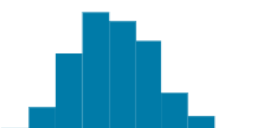
| Id                                                                                  | # SepalLengthCm                                                                     | # SepalWidthCm                                                                      | # PetalLengthCm                                                                     | # PetalWidthCm                                                                        | Species            |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------|
| SPL-SPW-PTL-PTW(CM)                                                                 | Length of the sepal (in cm)                                                         | Width of the sepal (in cm)                                                          | Length of the petal (in cm)                                                         | Width of the petal (in cm)                                                            | Species name       |
|  |  |  |  |  | 3<br>unique values |
| 1                                                                                   | 4.3                                                                                 | 2                                                                                   | 1                                                                                   | 0.1                                                                                   |                    |
| 150                                                                                 | 7.9                                                                                 | 4.4                                                                                 | 6.9                                                                                 | 2.5                                                                                   |                    |

However, the extended version significantly increases the number of samples, providing a richer and larger dataset that is more suitable for complex tasks like streaming and real-time clustering. With this larger dataset, we can better train models and handle real-world scenarios where data arrives continuously.

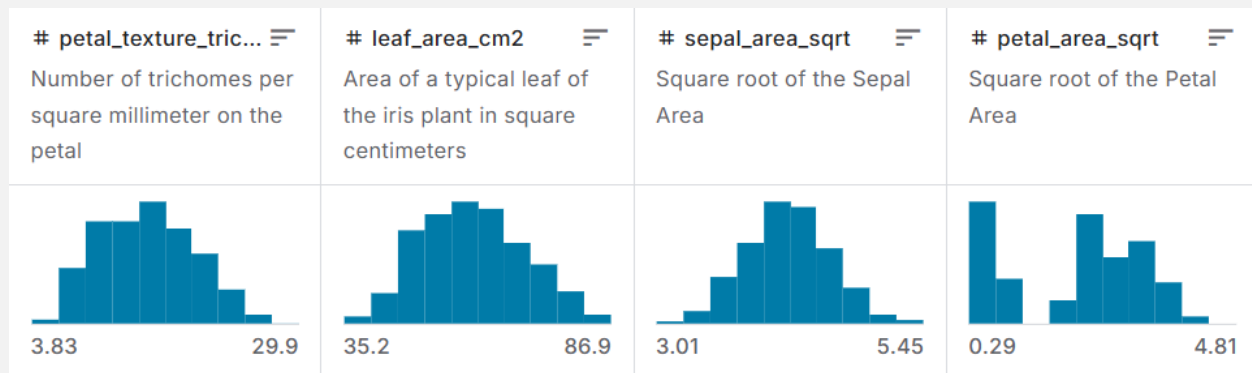
<sup>1</sup> <https://www.kaggle.com/datasets/samybaladram/iris-dataset-extended>



The dataset contains 1200 samples. Here, there are details of iris extended<sup>2</sup>:

|                                                                                                                                                                       |                                                                                                                                                                       |                                                                                                                                                                        |                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Δ species</b> <br>Species of the iris flower (Setosa, Versicolor, Virginica)      | <b># elevation</b> <br>Elevation level where the iris was found (in meters)          | <b>Δ soil_type</b> <br>Type of soil where the iris was found (Loamy, Sandy, Clayey) | <b># sepal_length</b> <br>Length of the sepal in centimeters                       |
| <b>3</b><br>unique values                                                                                                                                             | <br>50.1 300                                                                         | clay 37%<br>loamy 34%<br>Other (353) 29%                                                                                                                               | <br>4.04 9.12                                                                      |
| <b># sepal_width</b> <br>Width of the sepal in centimeters                           | <b># petal_length</b> <br>Length of the petal in centimeters                         | <b># petal_width</b> <br>Width of the petal in centimeters                          | <b># sepal_area</b> <br>Area of the sepal calculated as Sepal Length × Sepal Width |
| <br>1.7 4.76                                                                         | <br>0.96 7.84                                                                        | <br>0.05 2.99                                                                        | <br>9.05 29.7                                                                      |
| <b># petal_area</b> <br>Area of the petal calculated as Petal Length × Petal Width | <b># sepal_aspect_ratio</b> <br>Ratio of Sepal Length to Sepal Width               | <b># petal_aspect_ratio</b> <br>Ratio of Petal Length to Petal Width              | <b># sepal_to_petal_le...</b> <br>Ratio of Sepal Length to Petal Length          |
| <br>0.09 23.1                                                                      | <br>0.98 4.08                                                                      | <br>1.59 34.4                                                                      | <br>0.75 5.41                                                                    |
| <b># sepal_to_petal_wi...</b> <br>Ratio of Sepal Width to Petal Width              | <b># sepal_petal_lengt...</b> <br>Difference between Sepal Length and Petal Length | <b># sepal_petal_widt...</b> <br>Difference between Sepal Width and Petal Width   | <b># petal_curvature_...</b> <br>Measure of petal curvature in millimeters       |
| <br>0.84 55.8                                                                      | <br>-1.55 4.66                                                                     | <br>-0.34 4.56                                                                     | <br>3.31 17.9                                                                    |

<sup>2</sup> Images are credited under kaggle.com



## Splitting

We divide the dataset into two main parts. This task is done by running the *splitter.py* file. This code will get the whole dataset stored in the csv format and output two csv files, each described here:

### Train test

The first part is the **train-test split**, which is used to build and validate the initial k-means clustering model offline (about 80% of the whole dataset, equal to 960 samples). This allows us to define the initial cluster centers and evaluate the model's accuracy. The code that does this part is available at the *splitter.py* file.

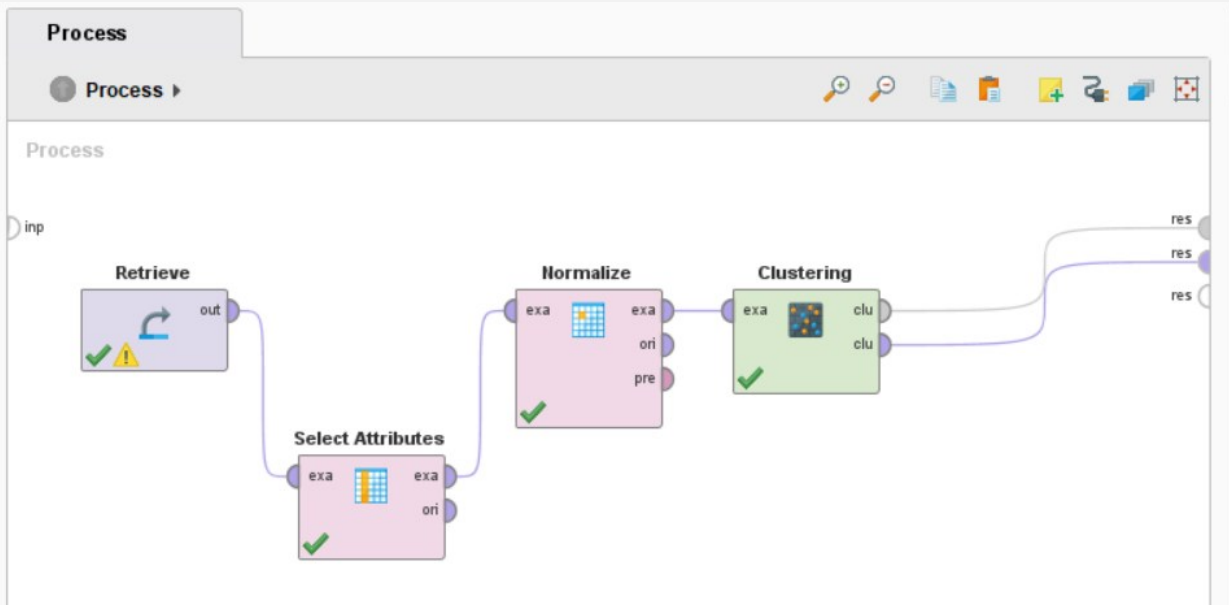
### Stream

The second part is the **streaming data**, which simulates a real-time environment by feeding the model new data continuously. As the new data arrives, the system dynamically assigns clusters and updates them over time, reflecting the changing nature of the incoming information. This split contains 20 percent of the whole dataset equal to 240 samples. The streamer code is available on the *streamer.py* file. We discuss further in the future about this code and its logic and implementation.

## Visualizing data (RapidMiner)

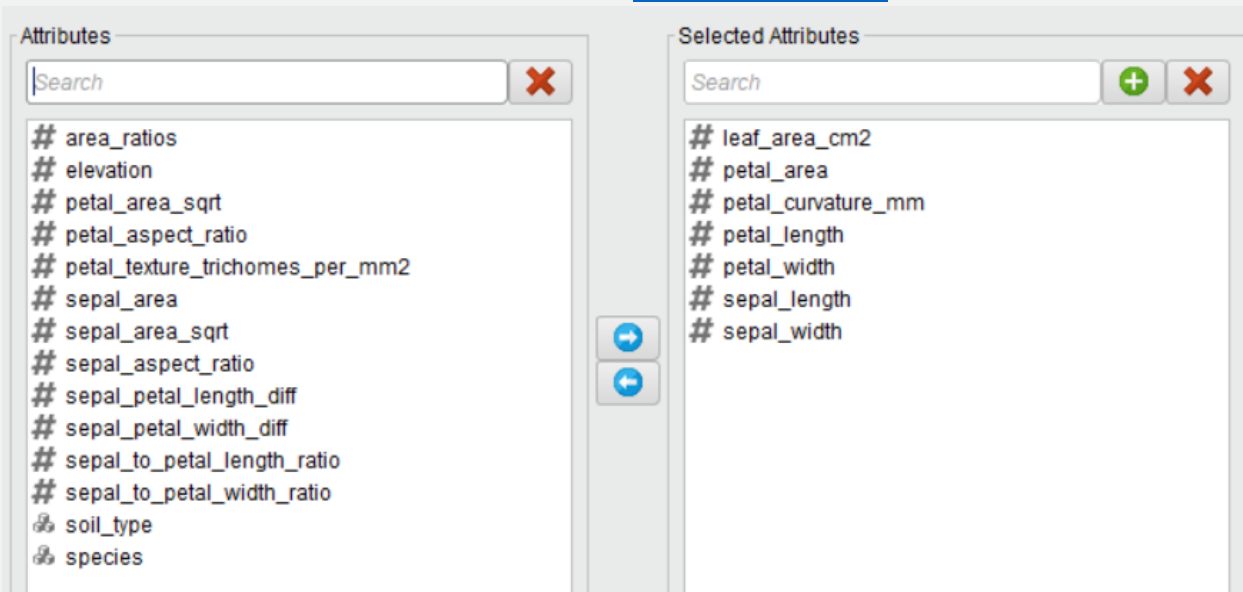
Using the RapidMiner application, we have trained some models and visualize them. Here is some information about this work:

The process is designed in this way.



At the first block, we load the dataset from file.

Then at the next block we choose some attributes to work with them. The selection of these features is described at [feature selection](#).

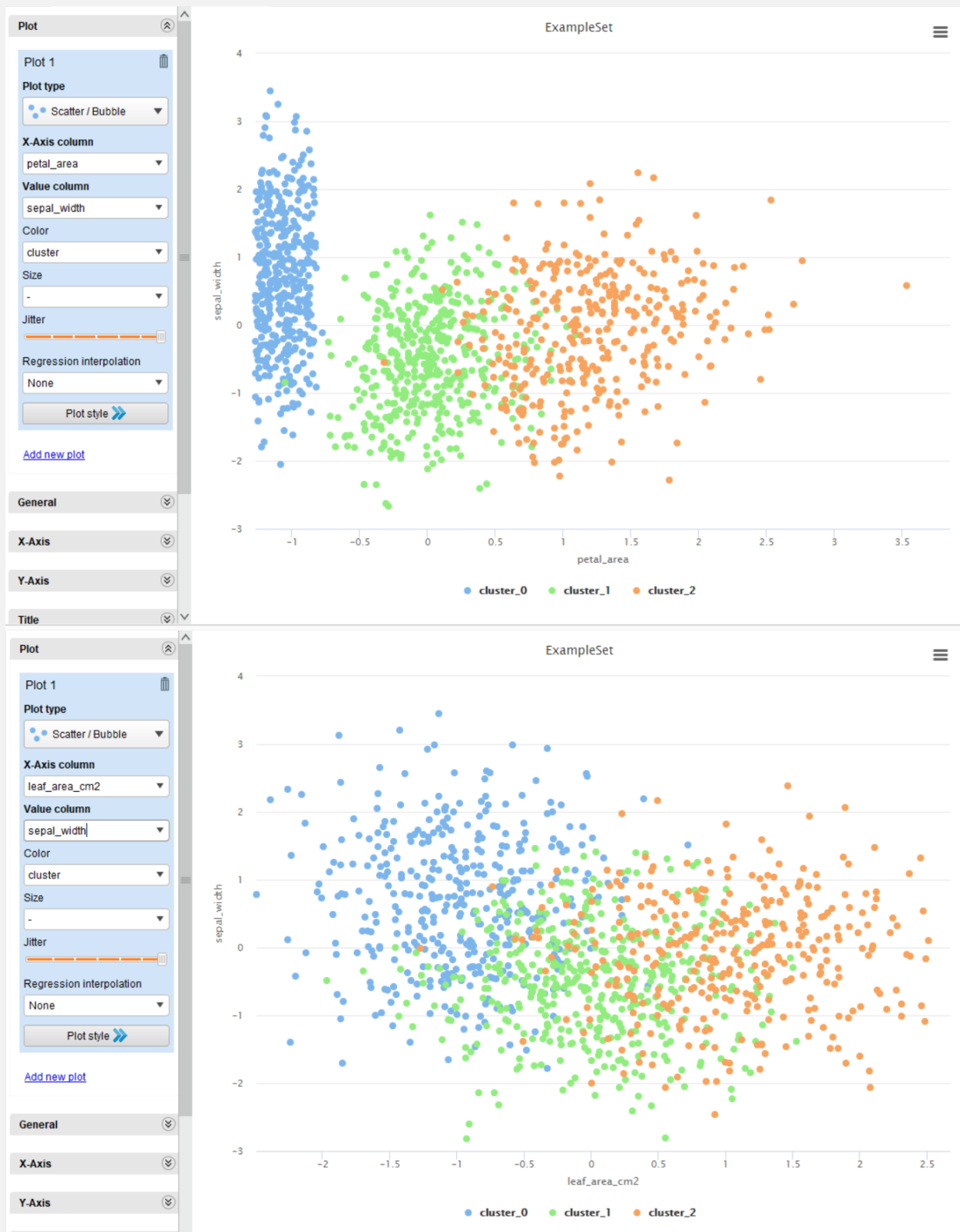


The next block is used for normalizing the data.

The last block does the clustering part; the value of  $k$  is set to 3, and max runs is set to the value of 25.

Here are some images of the visualization:





We can see the clustering algorithm clusters the data appropriately.

## *Training the model (Offline Phase)*

This part involves description for file *a.py* and will train the model for offline phase. The training of the model involves some steps that are described below:

### **Feature Selection**

The offline model, which the term offline refers to it because it is trained on the static data, not streaming data, is trained using *Kmeans* function imported from *pyspark.ml.clustering*.

For creating the model, we do not use all of the features in the dataset, not only because they make the processing too heavy, but also, because some of them are not meaningful. Thus, considering that the Kmeans algorithm struggles with high dimensional datasets, as described in our presentation, we had to eliminate some of the features before starting the training process. The features that we have used are:

|                    |              |               |              |
|--------------------|--------------|---------------|--------------|
| sepal_width        | sepal_length | petal_width   | petal_length |
| petal_curvature_mm | petal_area   | leaf_area_cm2 |              |

### **Standardize Features**

Standardizing the features is a crucial step in many machine learning algorithms, especially those based on distance measurements like k-means clustering. Standardization ensures that each feature contributes equally to the model by scaling the data to have a mean of zero and a standard deviation of one. Without standardization, features with larger ranges (e.g., petal length in centimeters) could dominate the distance calculation and bias the clustering process, making the model less effective.

For example, if one feature has values ranging from 1 to 100 while another ranges only from 0 to 1, the larger-range feature will have a much greater influence on how clusters are formed.

```
scaler = StandardScaler(inputCol="features", outputCol="scaledFeatures",  
                        withMean=True, withStd=True)  
scalerModel = scaler.fit(training_df)  
training_df = scalerModel.transform(training_df)
```

In the code, we use the *StandardScaler* to standardize the input features. The parameter *withMean=True* centers the data by subtracting the mean, and *withStd=True* scales the data by dividing by the standard deviation. The *scaler.fit(training\_df)* step computes the necessary statistics (mean and standard deviation), and the transform method applies the scaling to create the new *scaledFeatures*. This ensures that all features are on the same scale, improving the accuracy and reliability of the clustering by allowing the model to treat all dimensions equally. After this part, the data is ready for training.

## Training

The code trains a **K-Means clustering model** using the **scaled features** from the dataset. Here, the number of clusters is set to **k=3**, based on previous analysis using methods like the **silhouette score**. This method helps determine the performance of the model by considering the accuracy and the recall at the same time. We have described this evaluation metric in detail in our presentation.

The KMeans object is configured to use the column "scaledFeatures" as input, ensuring that the clustering process is not biased by differences in feature scales. The "prediction" column stores the cluster

assignments for each data point after training. The model is trained using the fit method, which iteratively adjusts the cluster centers and assigns data points until convergence is reached. The process is initialized with a random seed (seed=1) to

```
k = 3  
kmeans =  
KMeans(featuresCol="scaledFeatures",  
        predictionCol="prediction", k=k, seed=1)  
kmeans_model = kmeans.fit(training_df)
```



ensure reproducible results. This trained model can now be used to assign clusters to new streaming or test data.

## Evaluation

A Silhouette Score close to 1 indicates that the data point is well-matched to its own cluster and poorly-matched to neighboring clusters. A score of 0 indicates that the data point is on the border between two clusters, and a score of -1 indicates that the data point may have been assigned to the wrong cluster.

The Silhouette Score is useful for determining the optimal number of clusters in a dataset, as well as evaluating the quality of the clustering algorithm's performance. It can be used with a variety of clustering algorithms, such as K-Means or DBSCAN.

Mathematically, the silhouette score for a data point is calculated as:

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i)),$$

where:

- **a(i)** is the average distance between the data point and other points within the same cluster (intra-cluster distance).
- **b(i)** is the average distance between the data point and points in the nearest neighboring cluster (inter-cluster distance).

A high silhouette score indicates that clusters are dense and well-separated, making it a useful tool for determining the optimal number of clusters before finalizing the model.

The value of Silhouette Score for our training is 0.6174 which is a relatively good score.

```
hessam@mnt/hgfs/big2» python3 a.py [22:19:55]
25/02/07 22:21:48 WARN Utils: Your hostname, hessam-kk resolves to a loopback address: 127.0.1.1; using 192.168.134.131 instead
(on interface ens33)
25/02/07 22:21:48 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/02/07 22:21:51 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes wh
ere applicable
25/02/07 22:22:23 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
***** end training *****
Silhouette Score: 0.6174
hessam@mnt/hgfs/big2» [22:22:39]
```



## **Saving the Weights**

After training the model, we save the weights on file to use them at online streaming clustering. Since we use `random_seed` while training, the same training weights can be achieved by training.

## *Stream Clusterer (Online Phase)*

This model will load the pre-trained weights from the offline model which we previously described from file.

This part, which is implemented in file *d.py*, will load the weights, define the csv schema for receiving streams, receiving the streams from a tcp connection which is arriving from streamer program, process batches and cluster them in real time, evaluate them real time, and finally tune and partially fit the model based on arriving batches.

### **Loading the weights**

The model will load the weights that are saved on a file from the offline phase. This part does not have that much technical details, so we skip this part. It is just loading some values from file! :)

### **Defining CSV schema**

This will define the structure of the arriving streams so the model can handle it. We send the raw values of the samples without header for making it faster with lower overhead. However, in real world scenarios, the data can be received in JSON format with each values along with their corresponding header. That will format the samples.

### **Making connection for Streamer**

The clusterer will open a port on localhost:9999 and waits for a connection from streamer. This program is written to be able to handle connections from multiple sources using multithreading.

### **Streamer program**

The streamer program will send the samples in comma separated lines. They will be splitted into the parts after they arrived to the clusterer. The streamer will try to make a connection to localhost on port 9999 which online model opens that.

Activities Terminal 23:00 7 فوریه

\_\_\_\_\_

## Evaluation

```

Activities Terminal 00:03 8 قوربه python3 d.py
only showing top 20 rows
[+] Received from ('127.0.0.1', 51210): setosa
[+] Received from ('127.0.0.1', 51210): setosa
[+] Received from ('127.0.0.1', 51210): virginica
[*] Silhouette score for current batch: 0.6422
[+] Received from ('127.0.0.1', 51210): virginica
[+] Received from ('127.0.0.1', 51210): versicolor
[+] Received from ('127.0.0.1', 51210): versicolor
[+] Received from ('127.0.0.1', 51210): virginica (0 + 2) / 2
[+] Received from ('127.0.0.1', 51210): virginica
[+] Received from ('127.0.0.1', 51210): setosa (1 + 1) / 2
[+] Received from ('127.0.0.1', 51210): setosa
[+] Accuracy: 0.9565
[+] Recall: 0.9565
[+] F1-score: 0.9558
[+] Received from ('127.0.0.1', 51210): versicolor
[+] Received from ('127.0.0.1', 51210): setosa
[+] Received from ('127.0.0.1', 51210): virginica
+-----+
|species|prediction|scaledFeatures|
+-----+
|versicolor|1|[0.3454431236749232, -0.005712161713881428, 0.18341402039274463, 0.29043358744496744, 0.1716119868773682, 0.2044754815191039, -1.813834120023037]|
|setosa|0|[-0.9674572579529122, -1.1265627041482162, -1.360181716207571, -1.2178676858002495, -1.2398830441020139, -0.44603135873560973, 1.4226073096096072]|
|setosa|0|[-0.23345802277647745, -1.1042440173388846, -0.9711196760398685, -1.2854035637067518, -1.1165485268319708, -0.8315168937013663, 0.519414352502823]|
|virginica|1|[0.5193110447934325, 0.5930203292608236, 1.0659205993097287, 0.8419765903480695, 0.5553193793937245, 0.16833621261606396, -0.7789255233381794]|
|virginica|2|[0.5025357433819813, 0.7456401213215169, 1.761704908852072, 1.6130111964473037, 0.25383499830073037, -0.6749133951215279, -0.4778612043085853]|
|virginica|2|[0.7860462419638733, 0.6842093498307245, 2.27106399282432, 1.202167939182748, 0.4182810214174543, 0.011732714036225634, 0.801662151596929]|
|versicolor|1|[-1.180840615689265, 0.0861512051436312, 0.07586841579354234, 0.17224580110858853, 0.4045771861652273, 0.4333575179050211, -0.17679688526699022]|
|versicolor|1|[0.9361989011116766, 0.4207357293291396, -0.28472567021554773, 0.28480559761942575, 0.8019884084798107, -0.18101005344665264, 0.6887630319603447]|
|virginica|2|[1.0181934320937007, 1.8332084114963219, 1.992710662473443, 0.6731368955818139, 2.2545949452158736, 0.30084686526054305, 0.8392951914781422]|
|virginica|1|[0.32766117719689386, 0.27098734726730755, 0.268817882685822, 0.5155531804666419, 0.39087335091300024, 1.3970713553194114, -1.079989842377746]|
|virginica|2|[1.5081848461549554, 1.2102804665918152, 1.429677791364427, 1.0104442749398668, 1.130880454333259, 0.7706573610000581, 1.1027264706342879]|
|setosa|0|[-0.858624017611676, -1.086188939315741, -1.0122400542689753, -1.12781984850159, -1.1302523620041978, -1.3736059272469612, -0.026264725749192686]|
|versicolor|1|[-0.27396134530976696, 0.4472962718186076, -0.08861309712288403, 0.6562529261051803, 0.5142078681830434, -0.7833312018306408, -0.13916384538754084]|
+-----+
[+] Received from ('127.0.0.1', 51210): versicolor
[+] Silhouette score for current batch: 0.5710
[+] Received from ('127.0.0.1', 51210): setosa
[+] Received from ('127.0.0.1', 51210): virginica (1 + 1) / 2
[+] Received from ('127.0.0.1', 51210): versicolor
[+] Received from ('127.0.0.1', 51210): virginica (0 + 2) / 2
[+] Received from ('127.0.0.1', 51210): versicolor

```

In this example, we can see after receiving several samples from the streamer, the model evaluates the batch and gives the accuracy of 0.9565 and a recall of 0.9565, which results the F-1 score of 0.9558. The silhouette score for that specific batch is 0.5710.

On another batch, which the log of the streamer is also available at the screen shot at top right corner, the result of the evaluation metrics is as follows:

Accuracy: 0.96      recall: 0.9600      F1-Score: 0.9592

A problem which is shown at the image is that the stream processing takes some time, and that will prevent the new incoming samples to be evaluated. However, no data will be lost since we store all of them in the array, but the accumulation may cause the model to largely update the model weights. This may slightly affect the model accuracy if some batches come in order with a poor quality. We can implement a mechanism to control the updating logic, such as applying a lower learning rate for streaming data.





# MOA

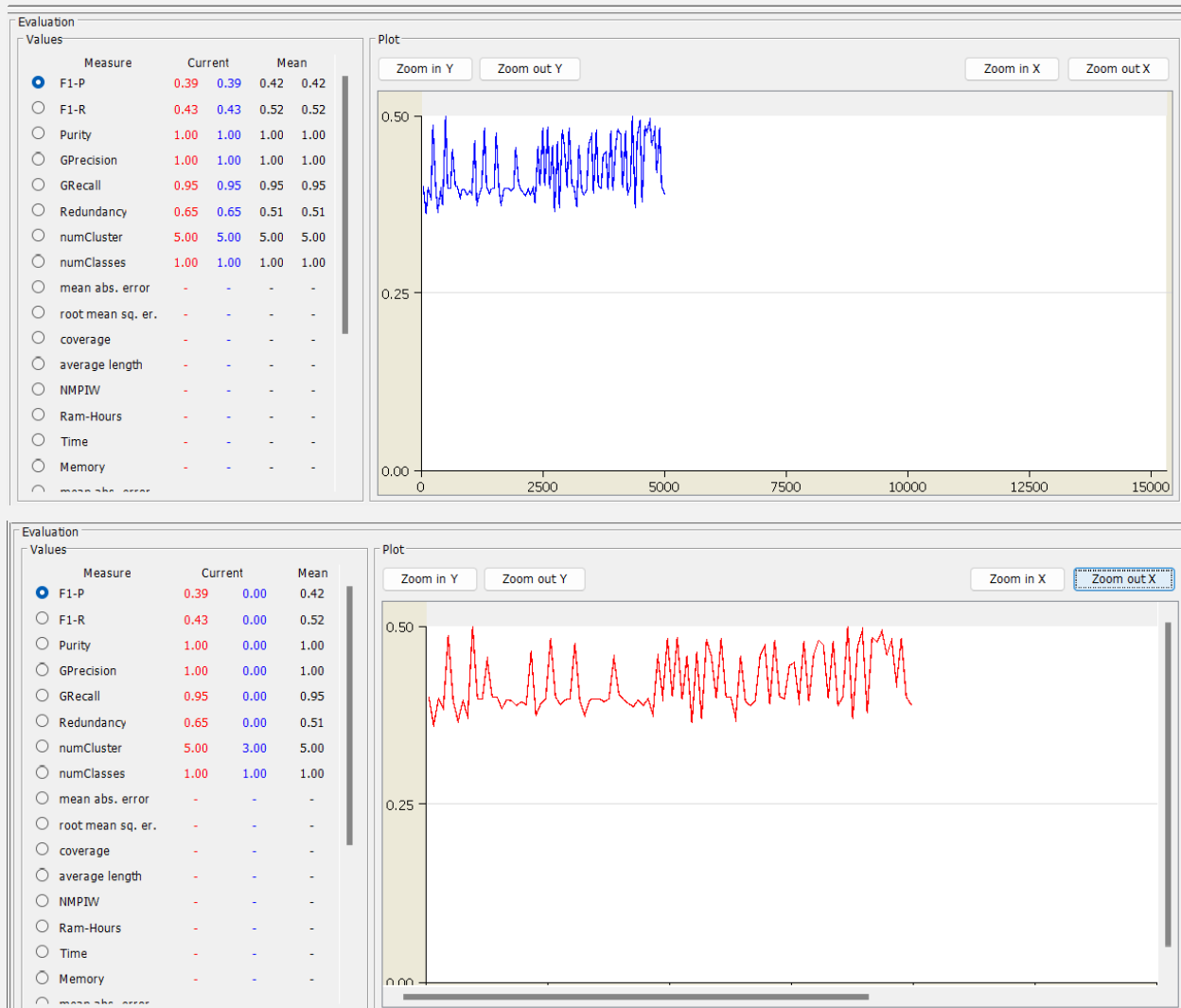
While working with these configurations on the MOA application, we could run different clustering algorithms on the data to see how they behave. Since MOA processes the data very fast and requires large amounts of data, we duplicated our dataset to make more samples.

The stream method on MOA was set to csv stream.

The first algorithm was set to StreamKM with default values for configurations.

The second algorithm which is used for comparison is set to different models each time.

One of the results that we have worked around with is shown at the image below:



## *Conclusion*

In this project, we demonstrated how to develop a system for real-time clustering of streaming data using k-means clustering. The key challenge was to handle continuously incoming data while ensuring the clustering model dynamically updates to reflect changes without sacrificing performance. To address this, we first trained an offline k-means model using a well-structured, extended Iris dataset, ensuring proper feature selection and standardization to enhance clustering accuracy. The offline model was designed to initialize cluster centers based on static training data, establishing a foundation for dynamic updates in the online phase.

During the streaming phase, we successfully simulated real-world scenarios by continuously feeding new data points through a custom streamer program. The model was able to assign clusters, evaluate performance, and update itself based on arriving batches of data. Evaluation metrics, including accuracy, recall, F1-score, and silhouette score, were computed to assess the effectiveness of the system. Results showed high performance, with the silhouette score consistently around 0.61 and accuracy exceeding 95%, demonstrating that the model remained robust and reliable even as new data streamed in.

We also highlighted some challenges, such as potential delays in processing larger batches due to random data arrival intervals. This can lead to accumulated data that, if not properly managed, could affect model accuracy. To address this, we proposed implementing mechanisms like learning rate adjustments for streaming updates to prevent large shifts in cluster assignments from poor-quality batches.

Overall, this project illustrates the practical application of clustering on streaming data, with key takeaways including the importance of proper offline training, dynamic tuning in the online phase, and continuous evaluation. The system is scalable and adaptable, making it applicable to real-world tasks such as anomaly detection, pattern recognition, and real-time decision-making. Future improvements could involve experimenting with alternative clustering algorithms, optimizing and working with the streaming framework such as PySpark, and applying adaptive learning techniques to further enhance performance under various data conditions.